

I did from questions 1 to 12 in the cpp part and all questions in the java swing part.

1) What do we call a method that is not implemented in the base class, but only in the subclasses?

The method is a pure virtual function and the class is called abstract class.

2) It is not possible anymore to instantiate base class objects. Why is that ?

Because base class is an abstract class.

3) What is the characteristic property of POO that allows it to create a vector with photos and videos and treat all of them in the same way ?

Polymorphism. This allows us to manipulate objects through the base class interface, but the correct method implementation is called at runtime depending on the real object type.

4) What is needed specifically in C++?

In cpp we explicitly need to change objects to virtual, in Java it is virtual by default.

5) What are the types of elements in the vector? Compared to Java.

It must be pointers because there would be object slicing and polymorphism would be lost.

In java, objects are always manipulated through references and methods are virtual by default.

6) Le groupe ne doit pas détruire les objets quand il est détruit car un objet peut appartenir à plusieurs groupes (on verra ce point à la question suivante). On rappelle aussi que la liste d'objets doit en fait être une liste de pointeurs d'objets. Pourquoi ? Comparé à Java.

A group must not destroy the objects it contains because the same object can belong to several groups.

If a group deleted its objects, other groups would keep dangling pointers, leading to undefined behavior.

Therefore, a group does not own the objects; it only references them.

For this reason, the list must store pointers to objects, not the objects themselves.

Using pointers also avoids copying objects and preserves polymorphism.

If objects were stored by value, object slicing would occur and derived class behavior would be lost.

With pointers, virtual methods are correctly dispatched at runtime.

This design is similar to Java, where collections store references to objects.

In Java, objects are not destroyed when a collection is destroyed.

Instead, the garbage collector deletes objects only when no references remain.

When we have objects with instance variables that are pointers, the copies can cause problems like double free.

The solution is to follow the Rule of Three. That is to say, always make the destructor, copy constructor and copy assignment operator.

This way we guarantee a deep copy.